PLACEGIVER

DOCUMENTATION

Author: Ulises Anibal Baamonde Baudo

Placegiver - Technical Documentation

Intermodular Project DAM - Course 2025/2026

Author: Ulises Anibal Baamonde Baudo
Cycle: Multiplatform Application Development
Course: 2025/2026

Table of contents

1 . System Architecture
2 . API
3 . Database
4 . Web Application
5 . Mobile Application
6 . Desktop Application
7. Security
8. Deployment
9. Testing
10. Usage and Maintenance Guide
11. Future Improvements

---

1 . System Architecture

Placergiver is a social network that allows developers in the video game industry to connect with each other.

The architecture consists of three main platforms connected to a database through a REST API:

```
Web App          ------------------------------------------
(PHP, Laravel)                                            |
                                                          □

Mobile App   ----------------------------->          Rest API (Java) -----------> Database
(Kotlin, Java)                                                                    (MySQL)
                                                          ↑
                                                          |
Desktop App                                               |
(C#, Windows Forms) -----------------------------------
```

Main Technologies

Component	Technology
Rest API	Java, JAX-RS (Jersey)
Database	MySQL
Mobile App	Kotlin/Java
Desktop App	C#, Windows Forms
Web App	PHP, Laravel
API Build	Maven
Android Build:	Gradle

2 . API

Endpoints List

Users

Endpoint	Method	Function
/usuarios?nombre	GET	Get user by a name
/usuarios/login?nombre&pass	GET	Authenticate user
/usuarios/registro	POST	Register a new user
/usuarios/actualizar	PUT	Update user data

Posts

Endpoint	Method	Function
/posts	GET	Get all posts
/posts/categoria	GET	Get posts by category
/posts/{nombreUsuario}	GET	Get posts by user
/posts/publicar	POST	Create a new post
/posts/publicarConCategoria	POST	Create a post with category
/posts/{idPost}	DELETE	Delete a post

Authentication

Currently, the API does not implement a robust authentication system. However, adding one is planned as a future improvement.

Request and Response Examples

- **Register User:**

  Request:

  POST https://localhost:8080/placegiver/usuarios
  Content-Type: application/json

```
{
        “usuario”: “Ulises”,
        “description”: “”,
        “email”: “ulises@example.com”,
        “password”: “examplePassowrd”

}
```

Response:

```
{
        “success”: true,
        “filas”: 1
}
```

- **Create Post:**

  Request:

  POST https://localhost:8080/placegiver/posts/publicar
  Content-Type: application/json

```
{
        “id”: 1,
        “texto”: “Hola”,
        “idCategoria”: 3,
        “usuario”: “Ulises”

}
```

  Response:

```
{
        “success”: true,
        “filas”: 1
}
```

- **Get All Posts:**

  Request:

  GET https://localhost:8080/placegiver/posts

Response:


```
{
    [
        {
            “id”: 1,
            “texto”: “Hola”,
            “idCategoria”: 3,
            “usuario”: “Ulises”,
            “fechaCreacion”: “27-02-2026”
        }
    ]
}
```


3 . Database

Entity-Relationship Diagram

```
+-------------+                +---------------------+                                  +------------------+
| USUARIOS|                    | PUBLICACIONES |                                        | CATEGORIAS |
+-------------  +               +-----------------------+                                +--------------------+
| nombre      | <----+         | id                      |       ------------------------->  | id                 |
| descripcion |          |     | texto                   |       |                           | nombre           |
| email       |  |       |     | fechaPublicacion  |       |                          +--------------------+
|  password  |          + --+|  usuario              |       |
  ----------------              |  idCategoria          | + ---
                                +---------------------+
```

Table and Fields Description

Users

Field	Type	Description
nombre	VARCHAR(20)	Unique username

descripcion	VARCHAR(500)	User profile description
email	VARCHAR(320)	User email
password	VARCHAR(20)	User password
fechaCreacion	DATETIME	Creation date

Posts

Field	Type	Description
id	int(11)	Primary key
texto	VARCHAR(500)	Post content
fechaPublicacion	DATETIME	Creation date
usuario	VARCHAR(500)	User name who made the post (FK to users)
idCategoria	int(11)	Category where the Post belongs to (FK to Categories)

Categories

Field	Type	Description
id	int(11)	Primary key
nombre	VARCHAR(50)	Category name

Important Rules

- **Usernames must be unique.** For each user in **users** table
- **Post's text cannot be null.** They must have some value
- **Every post must belong to a category.** The 'idCategoria' in the **posts** table cannot be null
- **Each post must belong to a single user.** The 'usuario' field cannot be null and defines that it was made by a unique user

Migration Management

The database is managed by MySQL that has a visual interface for designing tables and its own console for executing the necessary SQL.

4 . Web Application

Technologies:

- Composer - Dependency Manager
- Bootstrap - CSS Framework
- Laravel - Main Development Framework
- PHP/HTML/CSS - Main programming languages

The **project structure** is the common one used for projects in the Laravel Framework, with the most important parts being:

```
App
|
|- /app
|     |
|     |
|     ------ /Services -> API communication logic
|     |
|     ------/HTTP
|              |
|              | ------  Controllers -> Main Controllers to move data between the views
|
|
| - /resources
|              |
|              ------- css -> All CSS style files
|              |
|              ------- js → All javascript events for the app
|              |
|              ------- views -> Folder that contains all the pages for the app
|
| - /routes -> web.php -> Place where the main routes of the app are defined
|
|
|
```

```
|
|- .env -------> Connection configuration to the API
```

5 . Mobile Application

Technologies:

- Android Studio - Development Environment
- Java - Main Language
- Gradle - Build creation system
- Android SDK - Development Kit for Android Apps

The **project structure** is the common one used for projects of Android Studio, with the most important parts being:

```
app/
| --- app/src/main/
|   |
|   | ---- java/com/placegiver/
|   |   |—— Activities ----> Main App Activities (Main Activity, Login, Register, etc)
|   |   | ----- Fragments ->  Fragments that are part of the navigation on the MainActivity
|   |   |----- Adaptador- java-> Adapter for the Posts lists
|   |   |----- ApiRest.java -> Class with the different request to the api as functions
|   | ---- res/  ----> Contains the main resources for the app
```

6 . Desktop Application

Technologies:

- C# - Main programming language
- Windows Forms - Main Development Framework
- Components Library for Windows Forms - Creation of custom components for the app
- HTTP Client - Class used to connect to the API and make requests
- .NET 4.7.2 - Development platform

The **project structure** for the Desktop app combines two project one, the app by itself and the other one for the creation of custom components for the app:

```
APP
|
|---- /ComponentesRedSocial ---> Custom components used on the app
|---- /Resources ---> Main resources used on the app
|
| ---- FeedPrincipal.cs ------> Main page where the user can see the most recent post
| ---- IniciarSesion.cs ------> Form where the user can Log in with their account
| ---- CrearCuenta.cs ------> Form where the user can create their account
| ---- EditarPerfil.cs ------> Form used for edit the logged in user's account information
| ---- Perfil.cs ------> Used to show the profile of a user
```

7. Security

Security Mechanisms

- **Data Verification**: All the versions of the app verifies the fields before making any kind of Inserts or Update request to the API
- **SQL Injection**: A good part of the queries in the API used preparement statements

User and Permission Management

- Each user has a unique username
- Users can only edit their own profiles
- The user must be log in with their account to make posts
- Users can only delete their own posts

Good practices

- All fields are verified before requesting to the API
- All kind of problems or error with the requests are informed to the user
- Similar design for all platforms to make its use easy for everyone

8. Deployment

How to Run Each App

1. Mobile:

    1.1. Open Android Studio:
    1.2. Open the Placegiver project
    1.3. Run the app on physical device or on an emulator
2. Desktop:

    2.1. Open Visual Studio
    2.2. Open the Placegiver project
    2.3. Run the project

3. Web:
    3.1. Place the project on the PHP server folder (if you are using Xampp, put it on the htdocs folder)
    3.2. Open the developer console and navigate to the project folder
    3.3. Use the command ‘php artisan serve’ to run the app

API Deployment

For development purposes, the API normally uses localhost for its use, but it’s meant to have its own IP for the state of production. So, first of all, make sure to configure the Rest API and all the requests so they use the correct IP. After that, you need to:

1. Open Visual Studio Code:
2. Install the extension Community Server Connectors
3. Create a new Server, using Apache Tomcat 10.0.27 as the option to download.
4. Right click on the web.xml file and click on the “Run on Server” option
5. Choose the server recently created
6. Right Click on the Maven project and go to Run Maven Commands -> Deploy to create the WAR file
7. Right click on the Server and select ‘Add Deployment’
8. Add the WAR file that is located in the target folder
9. Initialize the server with the install and recompile options from the Lifecycle menu

Environments

As previously mentioned, the app uses mainly two environments, the local one its development and the computer's IP that hosts the app for its production:

- Local/Development: localhost:8080
- Production: 192.0.18.2:8080 (Not the real IP, just used as an example)

9. Testing

The manual tests made for the project are the following:

Users:

- Create a new user
- Log in with a valid user
- Failed to log in with wrong user's data
- Update user's profile

Posts:

- Create a post
- View a main's feed
- Filter posts by a certain category
- View posts from a certain user
- Delete a post

Other tests made for the app are:

- API connection: Verify that the connection with the API with the three apps works correctly without problems
- Current logs verification: Mess with the saved user data from the last session to confirm that the Apps can still run without problems

10. Usage and Maintenance Guide

Running Apps Locally

The requirements for running all the apps in your local system are:

- Composer
- PHP Server (Xampp recommended)
- Maven
- Java JDK 11 or higher
- Android Studio
- Visual Studio (2022 as a recommendation)

And the steps to run each of them are simple:

- Mobile App:
    - Open the project on Android Studio and run
- Web App:
    - Copy the project in the htdocs folder that is inside Xampp
    - Open the developer console and go to the project's folder
    - Execute 'php artisan serve' to run the page
- Desktop App:
    - Open the project on Visual Studio and run

If you desire to make any new features to the apps, you have to follow the structure of the project:

- API: Create new endpoints in the correct files depending of what kind of tables controls
- Datatable: Create new tables or fields if it doesn't mess with the rest
- Interfaces: Create new views, activities, etc, on the different apps respecting the structure of each project

11. Future Improvements

If the future of this project goes well, we plan to add all the initial proposed features to the add that couldn't be done the release, such as:

- Followers system
- Private Conversations (DM's)
- Comments on posts
- Likes List

Also, we would like to improve the app performance with:

- Implementation of prepared statements for all the queries of the Rest API
- Use of data encryption to improve the security